

University Student Groups

Scenario 1

A university has a social network containing 12 students. Some students form separate friendship groups.

Create an undirected graph with:

- Group 1: A, B, C, D
- Group 2: E, F, G, H
- Group 3: I, J, K, L

Use these relationships:

A-B
A-C
B-C
B-D
C-D

E-F
E-G
F-G
G-H

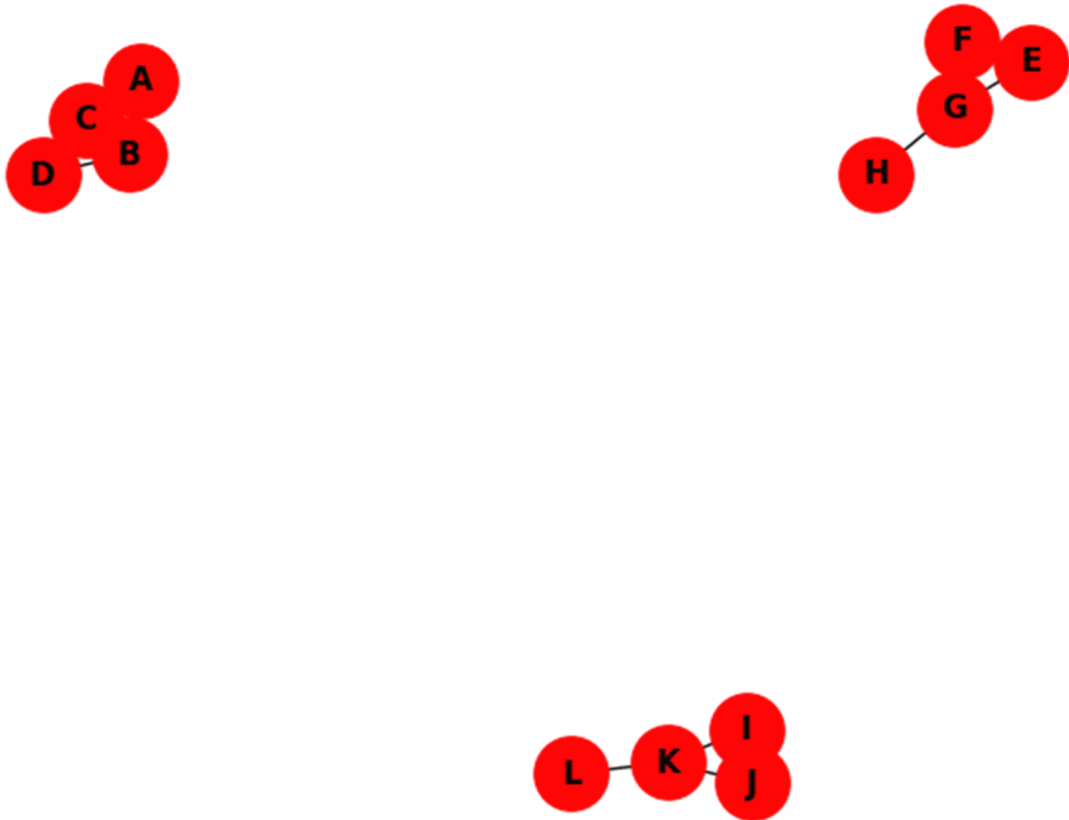
I-J
I-K
J-K
K-L

Tasks:

1. Create the graph using NetworkX.
2. Visualize it using Spring Layout.
3. Observe the graph carefully.
4. Determine how many separated groups appear in the visualization.
5. Find the connected components programmatically.
6. Print the nodes belonging to each component.
7. Calculate the number of nodes in each component.
8. Determine whether the entire graph is connected.
9. Identify which group contains the largest number of students.

Solution:

University Student Social Network



Number of nodes: 12
Number of edges: 13
Is graph connected? False

Connected Components:
Component 1: ['A', 'B', 'C', 'D']
Number of nodes: 4
Component 2: ['E', 'F', 'G', 'H']
Number of nodes: 4
Component 3: ['I', 'J', 'K', 'L']
Number of nodes: 4

Largest component: ['A', 'B', 'C', 'D']

Code:

```
import networkx as nx
import matplotlib.pyplot as plt

# Create an undirected graph
G = nx.Graph()
```

```

# Add edges
edges = [
    ("A", "B"),
    ("A", "C"),
    ("B", "C"),
    ("B", "D"),
    ("C", "D"),

    ("E", "F"),
    ("E", "G"),
    ("F", "G"),
    ("G", "H"),

    ("I", "J"),
    ("I", "K"),
    ("J", "K"),
    ("K", "L")
]

G.add_edges_from(edges)

# Spring layout
pos = nx.spring_layout(G, seed=42)

# Draw graph
nx.draw(
    G,
    pos=pos,
    with_labels=True,
    node_color="red",
    node_size=800,
    font_weight="bold"
)

plt.title("University Student Social Network")
plt.show()

# Number of nodes and edges
print("Number of nodes:", G.number_of_nodes())
print("Number of edges:", G.number_of_edges())

# Check connectivity
print("Is graph connected?", nx.is_connected(G))

# Find connected components
components = list(nx.connected_components(G))

print("\nConnected Components:")

for i, component in enumerate(components, 1):
    print(f"Component {i}: {sorted(component)}")
    print("Number of nodes:", len(component))

# Largest component
largest = max(components, key=len)

```

```
print("\nLargest component:", sorted(largest))
```

City Transportation Network

Scenario 2:

A transportation department wants to determine whether all areas of a city are connected by roads.

Create an undirected graph containing:

A, B, C, D, E, F, G, H, I, J

Roads:

A-B
A-C
B-C
B-D
C-D

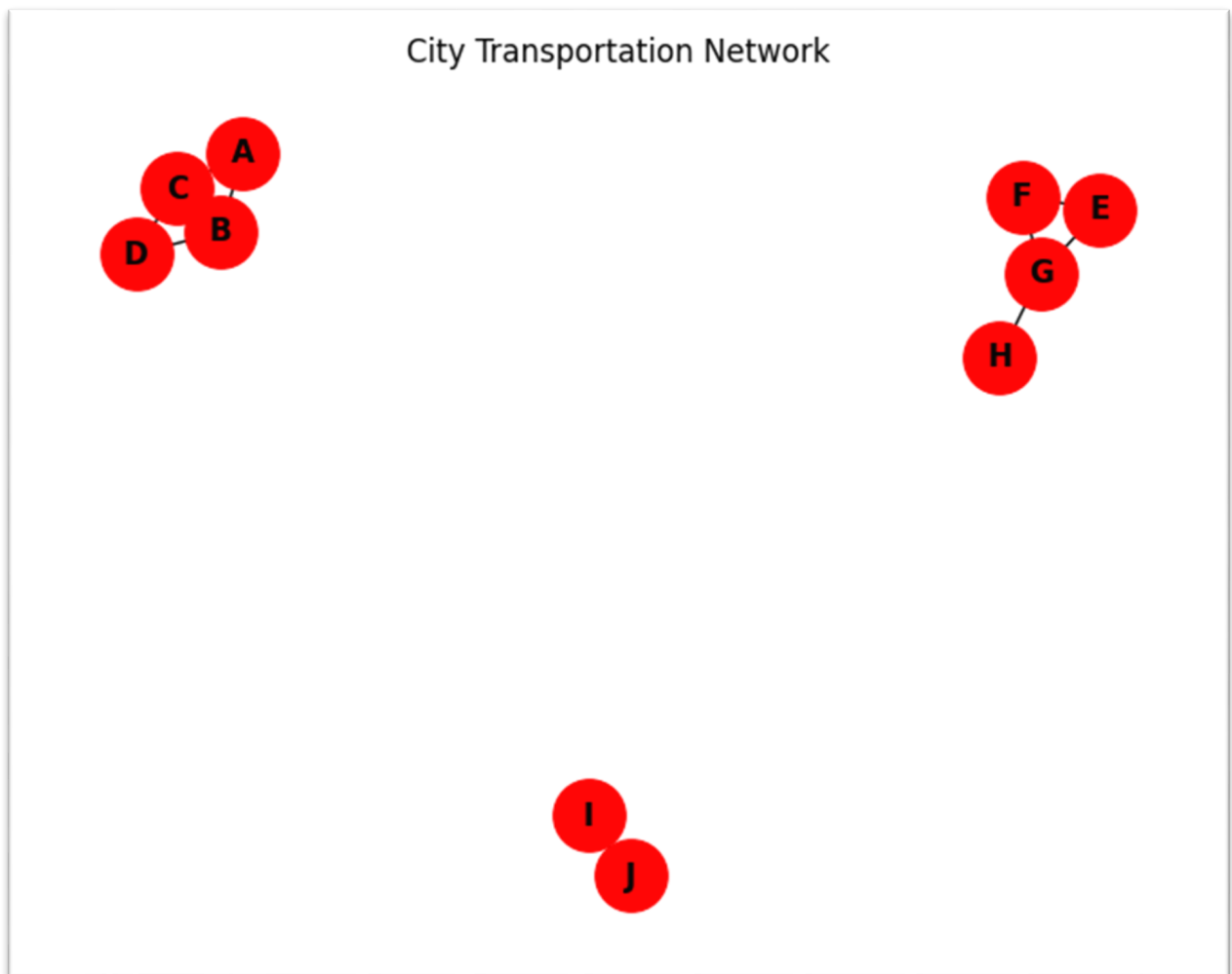
E-F
E-G
F-G
G-H

I-J

Tasks:

1. Draw the transportation network using `spring_layout`.
2. Identify visually separated parts of the network.
3. Find all connected components.
4. Print each component.
5. Print the number of nodes in each component.
6. Determine whether a vehicle can travel from A to J.
7. Determine whether the entire transportation network is connected.
8. Identify the isolated/smallest transportation component.

Solution:



Number of nodes: 10
Number of edges: 10
Is graph connected? False

Connected Components:
Component 1: ['A', 'B', 'C', 'D']
Size: 4
Component 2: ['E', 'F', 'G', 'H']
Size: 4
Component 3: ['I', 'J']
Size: 2

Largest transportation network:
['A', 'B', 'C', 'D']

Can A reach J? False

Code:

```
import networkx as nx  
import matplotlib.pyplot as plt
```

```

G = nx.Graph()

edges = [
    ("A", "B"),
    ("A", "C"),
    ("B", "C"),
    ("B", "D"),
    ("C", "D"),

    ("E", "F"),
    ("E", "G"),
    ("F", "G"),
    ("G", "H"),

    ("I", "J")
]

G.add_edges_from(edges)

# Spring layout
pos = nx.spring_layout(G, seed=42)

# Draw
nx.draw(
    G,
    pos=pos,
    with_labels=True,
    node_color="red",
    node_size=800,
    font_weight="bold"
)

plt.title("City Transportation Network")
plt.show()

# Basic information
print("Number of nodes:", G.number_of_nodes())
print("Number of edges:", G.number_of_edges())

# Connectivity
print("Is graph connected?", nx.is_connected(G))

# Components
components = list(nx.connected_components(G))

print("\nConnected Components:")

for i, component in enumerate(components, 1):
    print(f"Component {i}: {sorted(component)}")
    print(f"Size: {len(component)}")

# Largest component
largest = max(components, key=len)

print("\nLargest transportation network:")
print(sorted(largest))

```

```
# Can A reach J?  
print("\nCan A reach J?", nx.has_path(G, "A", "J"))
```

Hospital Departments

Scenario 3:

A hospital has 10 departments. Some departments communicate with each other, but some groups have no communication link with the rest of the hospital.

Departments:

Emergency, ICU, Surgery, Radiology,
Laboratory, Pharmacy, Cardiology,
Neurology, Administration, Finance

Connections:

Emergency-ICU
Emergency-Surgery
ICU-Surgery
Surgery-Radiology
Radiology-Laboratory
Laboratory-Pharmacy

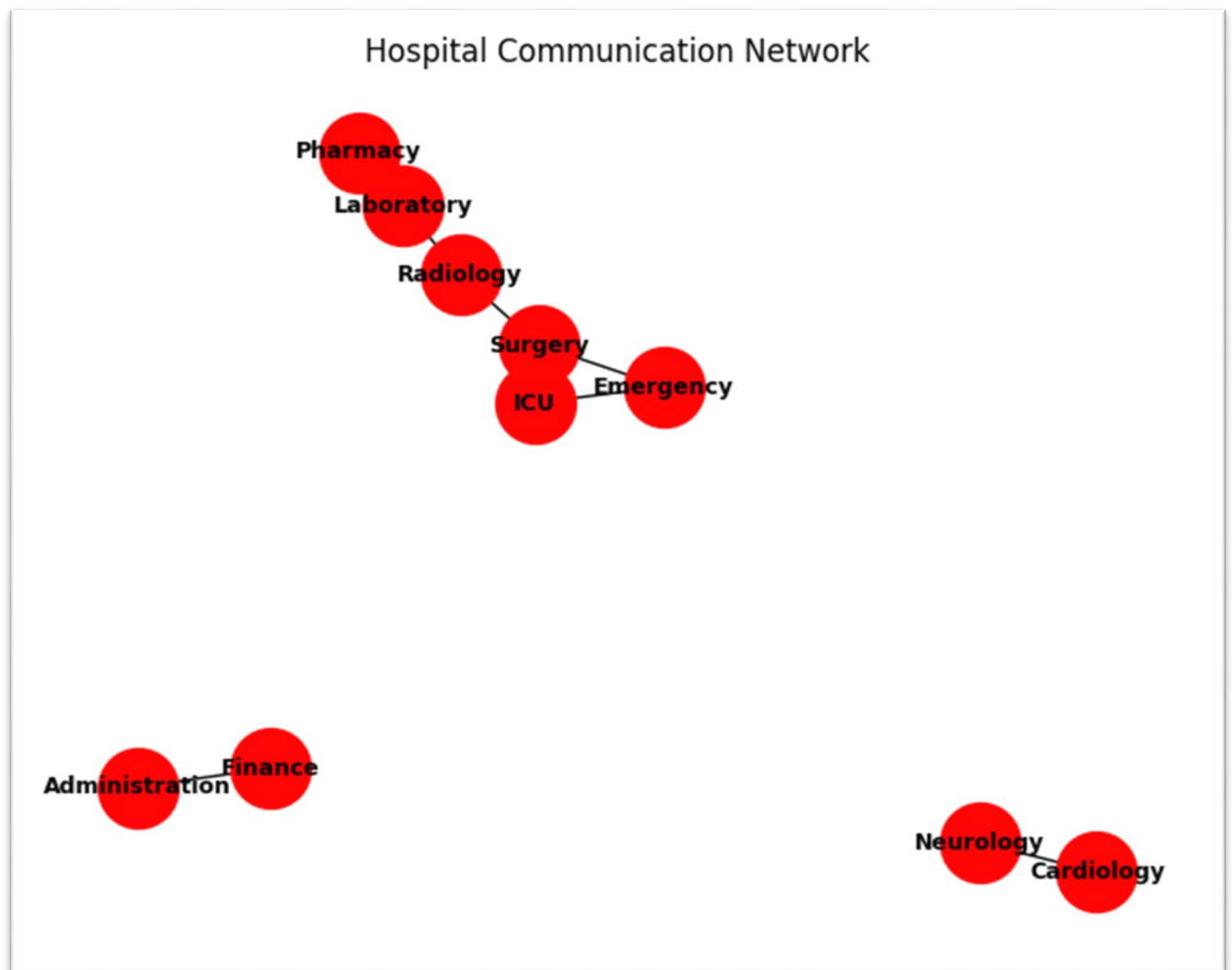
Cardiology-Neurology

Administration-Finance

Tasks:

1. Create the graph.
2. Visualize the graph using `spring_layout`.
3. Identify the visually separated groups.
4. Find all connected components.
5. Print the departments in each component.
6. Calculate the size of each component.
7. Determine which component contains `Emergency`.
8. Determine whether `Administration` can communicate with `Emergency`.
9. Determine whether the entire hospital communication network is connected.

Solution :



Number of nodes: 10
Number of edges: 8
Is graph connected? False

Connected Components:

Component 1:

['Emergency', 'ICU', 'Laboratory', 'Pharmacy', 'Radiology', 'Surgery']

Size: 6

Component 2:

['Cardiology', 'Neurology']

Size: 2

Component 3:

['Administration', 'Finance']

Size: 2

Largest component:

['Emergency', 'ICU', 'Laboratory', 'Pharmacy', 'Radiology', 'Surgery']

Can Emergency communicate with Administration? False

Code:

```

import networkx as nx
import matplotlib.pyplot as plt

G = nx.Graph()

edges = [
    ("Emergency", "ICU"),
    ("Emergency", "Surgery"),
    ("ICU", "Surgery"),
    ("Surgery", "Radiology"),
    ("Radiology", "Laboratory"),
    ("Laboratory", "Pharmacy"),

    ("Cardiology", "Neurology"),

    ("Administration", "Finance")
]

G.add_edges_from(edges)

# Spring layout
pos = nx.spring_layout(G, seed=42)

# Draw
nx.draw(
    G,
    pos=pos,
    with_labels=True,
    node_color="red",
    node_size=1000,
    font_weight="bold",
    font_size=9
)

plt.title("Hospital Communication Network")
plt.show()

# Basic information
print("Number of nodes:", G.number_of_nodes())
print("Number of edges:", G.number_of_edges())

# Connectivity
print("Is graph connected?", nx.is_connected(G))

# Components
components = list(nx.connected_components(G))

print("\nConnected Components:")

for i, component in enumerate(components, 1):
    print(f"Component {i}:")
    print(sorted(component))
    print("Size:", len(component))

# Largest component
largest = max(components, key=len)

```

```
print("\nLargest component:")
print(sorted(largest))

# Can Emergency communicate with Administration?
print(
    "\nCan Emergency communicate with Administration?",
    nx.has_path(G, "Emergency", "Administration")
)
```
